

SYSTEM HANDOVER · 交付文档

智能车牌识别 与分流系统

基于 Next.js + 百度大脑 OCR 的开放日车辆
实时识别、预约核验与门岗分流前端应用

项目代号

plate-ocr-app

技术栈

Next.js 16 · React 19 · SQLite

运行时

Node.js 24 (内置 node:sqlite)

文档版本

v1.0

交付日期

2026-06-04

目录

01 · 项目概览	1
02 · 系统架构与运作流程	2
03 · 技术栈与目录结构	3
04 · 部署与运行	4
05 · 页面路由说明	5
06 · API 路由说明	6
07 · 数据库设计	7
08 · CSV 导入格式规范	8
09 · 摄像头与扫描行为	9
10 · 注意事项与运维提示	10
11 · 常见问题排查	11
12 · 交付清单	12

1. 项目概览

本系统是为**校园 / 园区开放日车辆入场**设计的轻量级前端应用。门岗工作人员使用手机或平板，对准来访车辆车牌，应用通过**百度大脑车牌 OCR** 自动识别车牌号，再到本地 **SQLite 预约名单** 中核验，实时反馈“是否已预约 / 应分流到哪个停车区域”，并在大屏右上角以 HUD 实时显示**北一门 / 北二门**当前到场与预约总数对比。

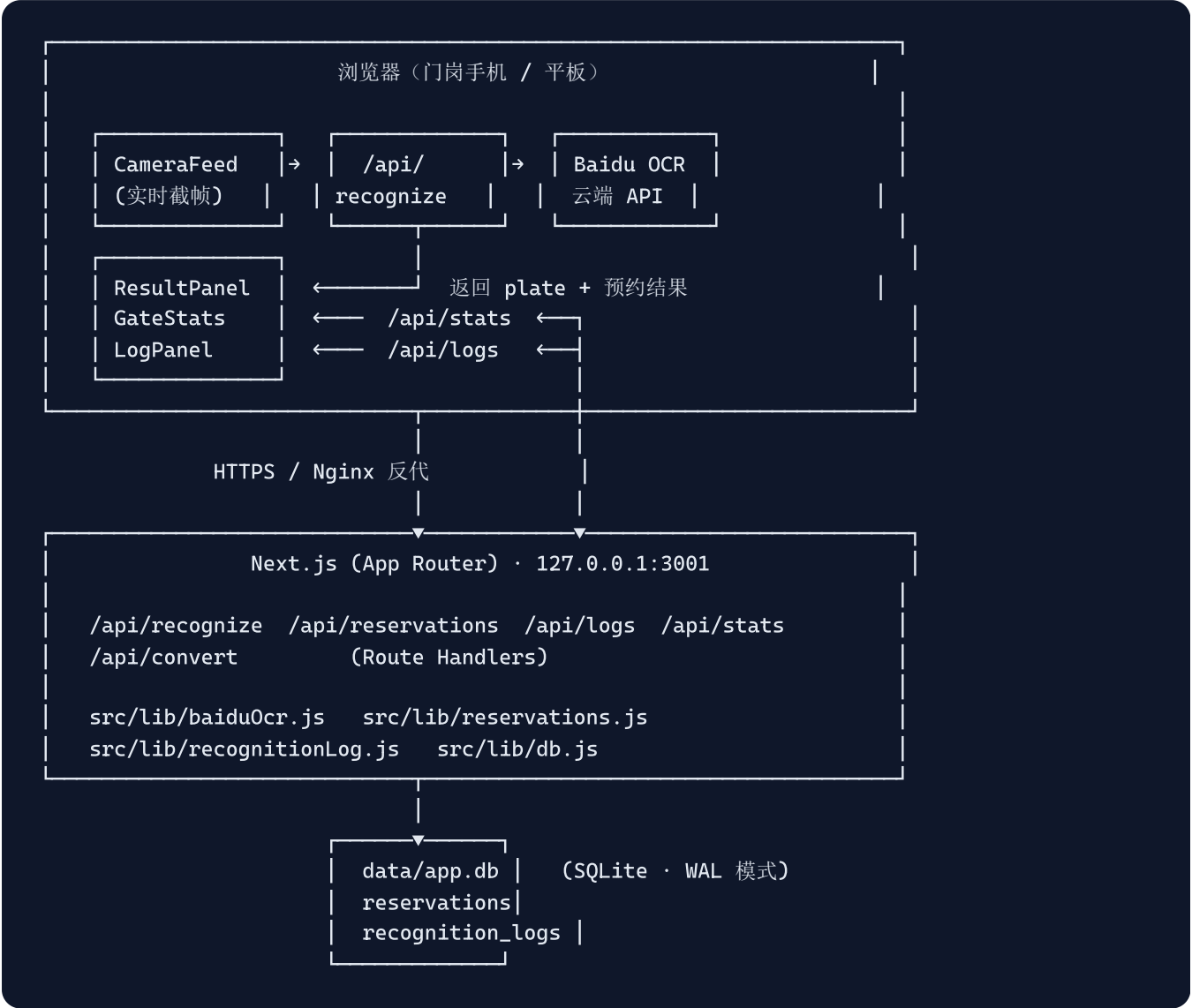
核心能力

- **实时扫描 / 拍照上传双模式**：高亮度场景实时取景识别，弱光或异常环境可改拍照上传。
- **车牌识别 + 预约核验一体**：识别成功后即在 SQLite 中查表，返回是否已预约及分配车区。
- **识别日志自动留痕**：每次成功识别（含手动查询）写入数据库，可在大屏抽屉或 `/peek` 后台查询。
- **门岗 HUD 实时统计**：右上角浮层显示「已到场 / 已预约」，达到预约总数时高亮提示。
- **CSV 一键覆盖预约名单**：支持 GBK / UTF-8 等编码，事务式整体替换。
- **后台管理** `/peek`：预约与日志的增删改查、清空。

面向角色：本文档面向**项目接收方的技术负责人 / 运维人员**，涵盖系统运作机制、所有路由的用途、部署方式、常见运维事项与排错。

2. 系统架构与运作流程

2.1 总体架构



2.2 一次"扫码—识别—分流"的完整流程

1. 用户进入 `/` (扫码大屏)，`CameraFeed` 申请摄像头并以 **1.5 秒一次** 的节奏抓取视频帧中心约 **86% × 1/3** 的车牌区域。
2. 抓帧后压缩到 $\leq 960\text{px}$ JPEG (质量 0.6)，转 Base64 发送到 `POST /api/recognize`。
3. 后端调用百度 OCR 接口 (`license_plate`)，归一化车牌 (去空格、去点、去横线、转大写)。
4. 在 `reservations` 表中按车牌精确查找，得到 `isReserved` 与 `parkingLot`。
5. 异步写入 `recognition_logs` (同一车牌已有记录时跳过，保留最早一条)。
6. 响应返回 `{ success, plate, isReserved, parkingLot }`，`ResultPanel` 立即展示结果并停止扫描，同时刷新 `GateStats`。
7. 门岗确认无误后点击"扫描下一辆车"，恢复扫描。

幂等设计：同一车牌多次识别只写入一次日志（按最早时间保留），统计中再以"车牌去重"方式计算"到场数"，避免同车反复扫描导致到场数虚高。

3. 技术栈与目录结构

类别	说明
前端框架	Next.js 16.2.6 (App Router) + React 19.2.4
样式	Tailwind CSS v4 + 少量内联样式 (/peek 后台使用对象样式)
字体	IBM Plex Mono (HUD 与车牌徽章需要的"终端感"等宽数字)
数据库	SQLite · 通过 Node 24 内置的 node:sqlite ，无原生编译
OCR	百度大脑 · aip.baidubce.com/rest/2.0/ocr/v1/license_plate
运行时	Node.js 24+ (必须，因依赖 node:sqlite)
进程管理	建议 PM2; HTTPS 经 Nginx 反代到 127.0.0.1:3001

3.1 目录结构 (已裁剪)

```
plate-ocr-app/
├─ data/
│   └─ app.db                # SQLite 主库 (WAL 模式)
│   └─ app.db-shm / app.db-wal # SQLite 共享内存 / WAL 日志
├─ recognition_logs/records.json # 旧版日志 (仅迁移用)
├─ public/                   # 静态资源
├─ src/
│   └─ app/
│       └─ layout.js          # 全局布局 (字体、metadata、viewport)
│       └─ page.js            # / 扫码大屏
│       └─ convert/page.js    # /convert CSV 转换上传页
│       └─ peek/page.js       # /peek 后台管理
│       └─ api/
│           └─ recognize/route.js
│           └─ reservations/route.js
│           └─ reservations/[id]/route.js
│           └─ logs/route.js
│           └─ logs/[id]/route.js
│           └─ stats/route.js
│           └─ convert/route.js
│   └─ components/
│       └─ CameraFeed.js      # 摄像头 + 截帧 + 上传
│       └─ ResultPanel.js     # 识别结果卡片 + 手动查询
│       └─ LogPanel.js        # 大屏抽屉式识别记录
│       └─ GateStats.js       # 右上角门岗 HUD 浮层
│   └─ lib/
│       └─ baiduOcr.js        # 百度 Token 缓存 + 车牌识别封装
│       └─ db.js              # SQLite 单例 + 建表 + 旧 JSON 迁移
│       └─ reservations.js    # 预约名单数据访问层
│       └─ recognitionLog.js  # 识别日志数据访问层
├─ .env.example
├─ next.config.mjs
└─ package.json
```

4. 部署与运行

4.1 环境准备

- **Node.js 24+** (必须, 依赖 `node:sqlite`, 低版本会启动报错)。
- 服务器需可外网访问 `aip.baidubce.com`。
- 申请**百度智能云车牌识别应用**, 获取 `API_KEY` 与 `SECRET_KEY`。

4.2 配置环境变量

项目根目录新建 `.env.local` :

```
BAIDU_OCR_API_KEY=your_baidu_ocr_api_key
BAIDU_OCR_SECRET_KEY=your_baidu_ocr_secret_key
```

4.3 启动命令

场景	命令	说明
本地开发	<code>npm ci && npm run dev</code>	默认 <code>http://localhost:3000</code>
生产构建	<code>npm run build</code>	构建产物到 <code>.next/</code>
生产启动	<code>npm run start:prod</code>	监听 <code>127.0.0.1:3001</code>
PM2 守护	<code>pm2 start npm --name plate-ocr -- run start:prod && pm2 save</code>	自动重启

生产环境注意：请勿使用 `npm run dev` 上线，必须 `build` 后再 `start:prod`，否则会开发模式运行，性能下降且暴露调试信息。

4.4 HTTPS 与 Nginx 反向代理

摄像头权限要求 **HTTPS 或 localhost**，必须把域名通过 Nginx 反向代理到 `127.0.0.1:3001`。最小可用配置示例：

```
server {
    listen 443 ssl http2;
    server_name your-domain.example.com;

    ssl_certificate      /path/to/fullchain.pem;
    ssl_certificate_key  /path/to/privkey.pem;

    client_max_body_size 10m;    # CSV / 图片上传留余量

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

5. 页面路由说明

扫码大屏 [GET /](#)

门岗主操作页面，包含：

- **摄像头视图**：实时取景 + 缩放（1× / 1.5× / 2× / 2.5×，自动优先硬件变焦）。
- **实时 / 拍照模式切换**：环境恶劣时改为相册或后置拍照。
- **结果卡片**：显示识别到的车牌、是否已预约、分配车区，并提供"手动查询车牌"输入框。
- **右上角 HUD (GateStats)**：两个门各显示 已到场 / 已预约，到达预约数自动高亮。
- **右下角浮动按钮** ：唤出抽屉式识别记录，可按车牌搜索、清空。

CSV 转换并覆盖预约名单 [GET /convert](#)

上传 CSV 文件，前端按所选编码（默认 **GBK**）解码并预览前 5 行；点击"确认无误，开始覆盖"后，将解码后的纯文本 POST 给 /api/convert，后端在事务中 **整体清空再批量插入** 预约名单。

后台管理 /peek GET /peek

非门岗页面，给运营人员看的数据后台，标签页切换：

- **预约名单**：新增、搜索、编辑（车牌 / 车区 / 已预约）、删除单条、**一键清空全部**。
- **识别日志**：搜索（按车牌前缀）、删除单条、**一键清空全部**。

⚠ **安全**：当前 /peek 没有任何身份认证，上线后必须通过 Nginx basic auth 或 IP 白名单保护此路径。建议同时保护 /api/reservations、/api/logs、/api/convert 的写操作。

6. API 路由说明

6.1 概览

方法	路径	用途
POST	/api/recognize	上传 Base64 图片，识别并核验预约
GET	/api/reservations	列出 / 模糊搜索 / 按车牌单查
POST	/api/reservations	新增一条预约
DELETE	/api/reservations	清空所有预约（事务）
PATCH	/api/reservations/[id]	更新单条预约
DELETE	/api/reservations/[id]	删除单条预约
GET	/api/logs	列出识别日志（支持 plate / limit）
POST	/api/logs	手动追加一条识别日志
DELETE	/api/logs	清空所有识别日志
DELETE	/api/logs/[id]	删除单条识别日志
GET	/api/stats	两个门的到场 / 预约统计
POST	/api/convert	解析 CSV 并整体覆盖预约名单

6.2 POST /api/recognize

请求体：

```
{
  "image": "data:image/jpeg;base64,/9j/4AAQ...",
  "source": "ocr"    // 可选，会写入日志
}
```

响应（识别成功）：

```
{ "success": true, "plate": "粤A12345", "isReserved": true, "parkingLot": "北一门" }
```

响应（未识别到车牌）：

```
{ "success": false, "message": "未能识别到车牌，请重新扫描" }
```

车牌会在入库与比对前统一归一化：去除空格、`·`、`.`、`-`，并转大写。

6.3 /api/reservations 系列

- **GET** /api/reservations?plate=粤A12345 → { success, reservation: {...} | null }
- **GET** /api/reservations?search=A12 → 列表模糊搜索（按 UPPER(plate) LIKE）
- **POST** /api/reservations Body: { plate, parkingLot, isReserved }
- **PATCH** /api/reservations/:id Body: 任意子集字段
- **DELETE** /api/reservations/:id 单条删除
- **DELETE** /api/reservations 一键清空（事务）

危险动作： **DELETE** /api/reservations 与 **POST** /api/convert 均会清空全部预约名单。**务必先备份** data/app.db。

6.4 /api/logs 系列

- **GET** /api/logs?plate=A12&limit=200
limit 默认 100，最大 1000；plate 为大写包含匹配。
- **POST** /api/logs Body: { plate, isReserved, parkingLot, source }，同一车牌已存在则不会重复写入，直接返回最早那条。
- **DELETE** /api/logs 清空全部日志
- **DELETE** /api/logs/:id 删除单条

6.5 GET /api/stats

按 parkingLot 字符串里是否包含「北一门 / 北门 / 北1门」「北二门 / 北2门」分组，输出：

```
{
  "success": true,
  "stats": {
    "北一门": { "arrived": 18, "reserved": 35 },
    "北二门": { "arrived": 9, "reserved": 20 }
  }
}
```

- **reserved**: 预约表中归属此门的总条数。
- **arrived**: 识别日志中 `isReserved=true` 且归属此门的**去重**车牌数。

6.6 POST /api/convert

接受 `multipart/form-data` (字段名 `file`) 或直接的 `text/plain` CSV 文本。解析后**事务式删除全部旧记录**并插入新记录, 返回插入条数。

```
{ "success": true, "message": "转换并覆盖成功", "count": 128 }
```

7. 数据库设计

数据库文件位于 `data/app.db`, 采用 SQLite WAL 模式。首次启动时由 `src/lib/db.js` 自动建表, 并把旧 JSON (`src/data/reservations.json` 与 `recognition_logs/records.json`) 一次性迁入, 迁移完成后写入 `meta.legacy_migrated` 标记, **不会重复迁移**。

7.1 表结构

```
-- 预约名单
CREATE TABLE reservations (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  plate       TEXT NOT NULL UNIQUE,           -- 归一化后的车牌
  parkingLot  TEXT,                          -- 分配车区（北一门 / 北二门 / NULL）
  isReserved  INTEGER NOT NULL DEFAULT 1,    -- 0 / 1
  createdAt   TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_reservations_plate ON reservations(plate);

-- 识别日志（同一车牌只保留最早一条）
CREATE TABLE recognition_logs (
  id          TEXT PRIMARY KEY,              -- 自定义 base36 id
  timestamp   TEXT NOT NULL,                -- ISO 8601
  plate       TEXT NOT NULL,
  isReserved  INTEGER NOT NULL DEFAULT 0,
  parkingLot  TEXT,
  source      TEXT                          -- ocr / manual / upload ...
);
CREATE INDEX idx_logs_plate      ON recognition_logs(plate);
CREATE INDEX idx_logs_timestamp ON recognition_logs(timestamp DESC);

-- 内部元数据（迁移标记等）
CREATE TABLE meta (
  key   TEXT PRIMARY KEY,
  value TEXT
);
```

7.2 备份与恢复

- 停服或低峰期直接复制 `data/app.db`（连同 `-wal` / `-shm`）即为完整备份。
- 使用 SQLite 自带 `VACUUM INTO 'backup.db'` 可在线热备份。
- 恢复：停服 → 替换 `data/app.db` → 重启应用。

8. CSV 导入格式规范

8.1 列约定

列序	含义	是否必填	说明
第 1 列	车牌号	必填	会自动去空格 / 点 / 横线并转大写；引号会被剥除
其余列	车区 / 状态标记	可选	任意列含 北一门 / 北门 / 北1门 → 北一门；含 北二门 / 北2门 → 北二门
其余列	未预约标记	可选	某列文本等于 false 或 否 时，标记为未预约

8.2 示例

```
车牌,车区
粤A12345,北一门
粤A23456,北二门
京B00001,北门
浙C99999,北2门,false
```

- 表头会被自动跳过（含 plate / Plate / 车牌 任一关键字）。
- 同一文件内重复车牌只保留第一条。
- 未指明车区时默认归为 北一门。
- Windows Excel 导出默认 GBK 编码，转换页默认即选 GBK；预览出现乱码时再切换 UTF-8 / GB2312 / Big5。

覆盖即清空： /api/convert 是**整体替换**语义，上传新 CSV 会清空数据库内全部预约。导入前请务必先备份 data/app.db 。

9. 摄像头与扫描行为

9.1 关键参数 (`src/components/CameraFeed.js`)

常量	含义	默认值
<code>REALTIME_INTERVAL_MS</code>	实时模式抓帧周期	1500 ms
<code>IMAGE_MAX_SIZE</code>	上传图最长边 (缩放阈值)	960 px
<code>JPEG_QUALITY</code>	上传 JPEG 质量	0.6
<code>ZOOM_LEVELS</code>	可选缩放档位	1, 1.5, 2, 2.5
<code>DEFAULT_ZOOM_LEVEL</code>	默认缩放	1.5×
请求分辨率	摄像头 <code>ideal</code>	1920 × 1080

9.2 截帧逻辑

每次只取视频帧**中心区域**: 宽度 $\approx$ 视频宽 $\times$ 86%, 高度取 `min(裁剪宽 / 3, 视频高  $\times$  50%)` 与镜头数字缩放共同决定, 典型车牌长宽比 $\approx$ 3:1, 能减少无关像素并降低上行带宽。

9.3 缩放策略

- 优先尝试 **硬件变焦** (`MediaTrackConstraints.zoom`), 支持的安卓机型会真正驱动镜头。
- 不支持时退化为 **CSS 数字缩放** (`transform: scale`), 仍可识别但清晰度受限。

9.4 锁屏 / 切后台恢复

代码监听了 `visibilitychange` / `pageshow` / `focus` 与 video 元素的 `pause` / `suspend` 事件: 回前台时若 track 已结束会重新 `getUserMedia`; 仅 video 暂停则主动 `play()`。这保证了门岗手机锁屏后回来无需手动刷新。

9.5 并发保护

`recognitionInFlightRef` 保证同一时刻只有一个识别请求在飞行, 避免百度 OCR 限流或重复扣费。

10. 注意事项与运维提示

10.1 路由未鉴权

当前所有 API 与 `/peek` 没有登录校验，**必须在 Nginx 层加上 IP 白名单或 Basic Auth 后再对外暴露**。写操作路径建议至少限制为：

- `/peek`
- `/convert`
- `/api/convert`
- `/api/reservations` (POST / PATCH / DELETE)
- `/api/logs` (POST / DELETE)

10.2 摄像头 = HTTPS

浏览器只允许在 `https://` 或 `localhost` 下访问摄像头。使用 IP 直连（`http://192.168.x.x`）时摄像头会报权限错误。

10.3 百度 OCR 配额

车牌识别接口有**每日免费额度**，上线前请确认配额并酌情充值。Token 在内存中缓存（提前 1 分钟过期），**重启后失效**，会自动重新申请。

10.4 日志去重规则

`appendRecognitionLog` 对**同一车牌只保留最早一条**，因此 `/api/logs` 不能用作“每次扫描的明细流水”，它表达的是“是否曾经识别到过此车牌”。统计 `/api/stats` 的“到场数”也按此口径去重。

10.5 数据库写入是单进程的

`node:sqlite` 在 PM2 cluster 多实例模式下会出现锁竞争，请**始终单实例运行**（`pm2 start ... -i 1`）。

10.6 数据迁移已自动完成

旧版 JSON（`src/data/reservations.json` 与 `recognition_logs/records.json`）会在**首次启动**自动迁入 SQLite，之后无需再保留这两个文件（保留也无副作用）。

10.7 摄像头分辨率请求是 `ideal`，并非强制

低端设备会自动降级。识别效果差时优先检查环境光照与是否启用硬件变焦。

11. 常见问题排查

现象	排查方向
启动报 <code>Cannot find module 'node:sqlite'</code>	Node 版本 < 24，请升级到 Node.js 24+。
页面提示「百度OCR配置缺失」	检查 <code>.env.local</code> 中是否填了 <code>BAIDU_OCR_API_KEY</code> / <code>BAIDU_OCR_SECRET_KEY</code> ，然后重启进程（环境变量不会热加载）。
「无法唤起摄像头」	1) 域名是否为 HTTPS；2) 浏览器站点权限是否被拒；3) 是否被系统级隐私设置屏蔽。
实时模式一直「扫描中」无结果	正常：识别失败不会停止扫描，只在控制台打印。请保证车牌进入中心裁剪框、亮度足够、对焦清晰。
识别车牌正确，但提示「未预约」	到 <code>/peek</code> 搜索该车牌确认是否真的在名单中；注意车牌会去掉空格 / 点 / 横线并转大写。
HUD 数字一直为 0	检查预约的 <code>parkingLot</code> 是否包含「北一门 / 北门 / 北1门 / 北二门 / 北2门」之一，否则不会被统计到任何门。
CSV 上传后车牌全乱码	在 <code>/convert</code> 顶部切换编码（GBK ↔ UTF-8 ↔ GB2312 ↔ Big5），观察预览区是否正常再确认覆盖。
误清空预约 / 日志	停服后用最近的 <code>data/app.db</code> 备份覆盖；若无备份，仅可从浏览器日志或百度 OCR 流水侧重建。

12. 交付清单

12.1 资产

- 源码仓库（含 `main` 分支）。
- 本文档 PDF（`docs/handover.pdf`）。
- 百度智能云车牌识别 `API Key` / `Secret Key`（线下传递）。
- 示例 CSV（`src/data/test_data.csv`）。

12.2 上线前 Checklist

- ☐ 服务器 Node.js 升级到 24+。
- ☐ `.env.local` 配置完毕并 `npm run build` 成功。
- ☐ PM2 守护并 `pm2 save`。
- ☐ Nginx 配置 HTTPS 反代到 `127.0.0.1:3001`。

- ☐ 给 `/peek`、`/convert` 等写操作路径加访问控制。
- ☐ 设置 `data/app.db` 的定期备份（建议每日 cron）。
- ☐ 在生产域名下完成一次「实时扫描 + 手动查询 + CSV 覆盖 + HUD 显示」端到端验证。

祝交付顺利。如有疑问，请优先对照本文档的「常见问题排查」一节定位，仍未解决再联系原开发同学。